

# Assignment 1

---

*Approach & Hints — how to think about each question*

*Linear Systems · Matrix Decompositions*

AIMLC ZC416 · Mathematical Foundations for Machine Learning  
M. Tech. in AI&ML

Each question — what it asks, how to approach it, and the structures behind it — shown  
with diagrams and worked intuition.

§ 0

## Before you start: a few ground rules

*Three things are worth settling up front: what ‘no built-in functions’ means, how to handle the random numbers, and how to present the work.*

### What ‘without built-in functions’ means

The rule targets functions that do the *mathematical work* for you. You implement the algorithms yourself; the language’s ordinary plumbing is fine.

- **Not allowed** — anything that directly returns the echelon form, a decomposition, a solution, rank, inverse, or determinant: `numpy.linalg.solve` / `inv` / `qr` / `matrix_rank`, `scipy.linalg.lu` / `cholesky` / `qr`, sympy’s `rref()` / `LUdecomposition()` / `cholesky()`, or the MATLAB / Octave equivalents `rref`, `lu`, `qr`, `chol`, `inv`, `A\b`.
- **Allowed** — ordinary constructs: loops, `range`, `len`, `enumerate`, indexing, slicing, `.append()`, list comprehensions, `abs()`, and a square root (`math.sqrt` or `**0.5`). Generating random numbers is expected. Using arrays purely as storage is fine — just don’t call their linear-algebra methods.

*Quick self-test: if your REF function is three lines long, you have called a built-in. A correct hand-written REF / RREF / LU / Cholesky / QR is roughly 15–25 lines of loops.*

### Random entries and numerical care

- Entries of  $A$  and  $b$  must be **system-generated** (instruction 3 — deterministic entries score nothing). Integers (`random.randint`) or reals (`random.uniform`) are both fine. The *dimensions* may be taken from the user; the *entries* must be random.
- Work in **floating point** throughout. Elimination naturally produces decimals ( $6/4 = 1.5$ ) — that is expected. In Python, `/` already gives floats; just avoid integer division (`//`) and `int()` casts.
- When checking for a zero pivot, do **not** test `== 0`. Round-off can leave something like  $1e-16$  instead of 0. Use a tolerance: treat `abs(value) < 1e-9` as zero. The ‘division by 0’ note in Q1(a) is handled by the row-swap step, or by choosing a different  $A / b$ .

### How to present each part

A three-part structure works cleanly for every question and is exactly what the deliverables ask for:

- **(1) A small hand-worked example** — e.g. a  $3 \times 4$  or  $3 \times 3$  matrix solved on paper, with brief explanation, to show you understand the method.
- **(2) The code snippet** — the function that performs the activity (neat handwriting or a clear screenshot). Only the relevant part, not the whole script.

- **(3) The output** — what your code prints, including the required-size run ( $6 \times 9$  for Q1c,  $7 \times 5$  for Q2d).

You do *not* need to hand-transcribe 20 elimination steps for a  $6 \times 9$  matrix. The code does the work; you show its input and output. The small hand example is what demonstrates understanding.

## § 1

## Q1 — Finding solutions of linear systems

The whole of Q1 is one pipeline: build the augmented matrix, reduce it, read off the solution structure. Parts (a) and (b) build the tools; part (c) runs them on a random  $6 \times 9$  system.

### Q1 (a) REF and RREF

In one sentence: reduce  $[A \mid b]$  to the staircase form (REF), then to the cleaned-up unit-pivot form (RREF), writing the elimination yourself.

*The idea.*

REF is the ‘staircase’ form: each row’s leading non-zero (its *pivot*) sits strictly to the right of the pivot in the row above, and everything directly below a pivot is zero. RREF goes one step further — each pivot is scaled to 1 and the entries *above* each pivot are cleared too, so every pivot column becomes a unit column.

|   |   |   |   |   |
|---|---|---|---|---|
| p | * | * | * | * |
| 0 | p | * | * | * |
| 0 | 0 | p | * | * |
| 0 | 0 | 0 | 0 | 0 |

→

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 0 | 0 | * | * |
| 0 | 1 | 0 | * | * |
| 0 | 0 | 1 | * | * |
| 0 | 0 | 0 | 0 | 0 |

REF (left): pivots  $p$  step down-right, zeros below. RREF (right): pivot columns become unit columns; free columns (marked  $*$ ) keep their coefficients.

*Approach.*

- **REF (forward elimination).** Keep a current pivot row. For each column, look for a non-zero entry at or below the pivot row; swap it up; if the whole column below is zero, move on. Use the pivot to clear every entry below it ( $\text{factor} = \text{entry} / \text{pivot}$ , then subtract  $\text{factor} \times \text{pivot-row}$ ). Advance the pivot row.
- **RREF (from REF).** Divide each pivot row by its pivot so the pivot becomes 1, then clear the entries *above* each pivot the same way you cleared below.

*Pitfalls.*

Float arithmetic with a tolerance for the zero test (see ground rules). The division-by-zero case is exactly why the pivot search and row swap exist.

## Q1 (b) Pivot / non-pivot columns, particular and null-space solutions

**In one sentence: from the RREF, name the pivot and free columns, then write one solution to  $Ax = b$  and a basis for all solutions of  $Ax = 0$ .**

*The idea.*

- A **pivot column** contains a leading 1 in some row; a **free column** does not. Free columns are the free variables — you may set them to anything.
- **Particular solution.** The simplest choice: set every free variable to 0. Then each pivot variable just equals the right-hand-side entry sitting in its row.
- **Null space** ( $Ax = 0$ ). Each free variable gives one basis vector: set that free variable to 1, the other frees to 0, and read off the pivot variables as the negatives of that free column's entries in the pivot rows.

A concrete look at that last rule. Suppose, after RREF, a pivot row reads

$$x_1 + 2x_4 = 0 \quad (\text{with } x_4 \text{ a free variable}).$$

Setting  $x_4 = 1$  forces  $x_1 = -2$  — which is exactly ‘minus the entry of the free column in that pivot row’. Do this for every free variable and you have one null-space vector each.

- **General solution.**  $x = x_p + \alpha_1 n_1 + \alpha_2 n_2 + \dots$  — the particular solution plus any combination of null-space vectors.

*With  $m < n$  you have at most  $m$  pivots and at least  $n - m$  free columns — so there are infinitely many solutions: a family with  $(n - r)$  free parameters, where  $r$  is the number of pivots.*

Geometrically, the solution set is a flat (an affine subspace): one point  $x_p$  that satisfies the system, shifted along the directions  $n_1, n_2, \dots$  that the matrix sends to zero.

## Q1 (c) The random $6 \times 9$ system, end to end

**In one sentence: run everything from (a) and (b) on a randomly generated  $6 \times 9$  matrix, and verify the general solution.**

*What is fixed and what is random.*

The size is **fixed at  $6 \times 9$**  — do not ask the user for it. Only the *entries* of  $A$  are randomly generated by your code (not typed in row by row).  $6 < 9$ , so expect at most 6 pivots and at least 3 free columns — a three-parameter family of solutions.

*Choosing  $b$  — the one real trap.*

If you draw  $b$  at random, independently of  $A$ , the system is almost always **inconsistent** — no solution exists, and the ‘particular solution’ you read off would not actually satisfy  $Ax = b$ . The clean fix:

Draw a random vector  $x_{seed}$  and set  $b = A \cdot x_{seed}$ . Now  $b$  lies in the column space of  $A$  by construction, so the system is guaranteed consistent and a solution exists.

*Verifying the general solution.*

Pick random coefficients  $\alpha$  for the free variables, form  $x = x_p + \sum \alpha_k n_k$ , and check that  $A \cdot x$  equals  $b$  (within tolerance). It works for *every*  $\alpha$  because each  $A \cdot n_k = 0$ , so  $A \cdot x = A \cdot x_p = b$ . Reporting the residual  $A \cdot x - b \approx 0$  is the verification.

*What to show.*

The random  $A$  and  $b$ , then REF, RREF, pivot columns, non-pivot columns, the particular solution, the null-space basis, the general solution, and the residual that verifies it.

§ 2

## Q2 — Matrix decompositions

Each part factors a matrix into simpler pieces. The conceptual note ‘Three Matrix Decompositions’ explains the ideas; here we focus on what each question specifically wants, and how it matches the lecture.

### Q2 (a) LU via elementary matrices

**In one sentence: eliminate  $A$  to upper-triangular form, but realise each row operation as an actual matrix — exactly the construction from Lecture 3.**

*What it asks — read this carefully.*

Take an SPD matrix  $A$  ( $n \times n$ ). For **every elementary row operation** you perform during elimination, **build the corresponding elementary matrix**. Use those to write  $A = LU$ .

*Note carefully: the elementary matrices are themselves the deliverable. Storing multipliers directly in  $L$  (the Doolittle shortcut) skips them and answers a different question — build each elementary matrix explicitly.*

*The idea (exactly as in Lecture 3, slides 24–28).*

An elementary matrix is the identity with one entry below the diagonal made non-zero. The lecture’s own example: to subtract 2×row 1 from row 2, multiply on the left by

|    |   |   |
|----|---|---|
| 1  | 0 | 0 |
| -2 | 1 | 0 |
| 0  | 0 | 1 |

$$\cdot A = (\text{row 2 becomes } R2 - 2 \cdot R1)$$

One elementary matrix  $E$  — identity with a single  $-2$  (shaded) at position  $(2, 1)$ , the operation ‘ $R2 := R2 - 2 \cdot R1$ ’.

Apply one such  $E$  per elimination step. Then, as the lecture shows, the whole elimination is

$$E_m \cdots E_1 A = U \Rightarrow A = E_1^{-1} \cdots E_m^{-1} U = L U$$

Each  $E^{-1}$  is the identity with the sign of its off-diagonal entry flipped; the product of these inverses is the lower-triangular  $L$ . (The lecture writes this compactly as  $L^{-1}A = U$ .)

|   |   |   |
|---|---|---|
| * | 0 | 0 |
| * | * | 0 |
| * | * | * |

×

|   |   |   |
|---|---|---|
| * | * | * |
| 0 | * | * |
| 0 | 0 | * |

=

|   |   |   |
|---|---|---|
| * | * | * |
| * | * | * |
| * | * | * |

*The goal:  $A = L$  (lower-triangular)  $\times$   $U$  (upper-triangular).*

### Approach.

- Generate an SPD  $A$  safely, e.g.  $A = B^T B + nI$  for a random  $B$ : symmetric by construction, positive-definite because of the  $+nI$ . SPD also guarantees no zero pivots (Lecture 3, slide 8), so you

never need row swaps — every elementary matrix is the simple ‘add a multiple of one row to another’ type.

- Start  $U = A$ ,  $L = I$ . For each below-diagonal position, compute  $f$ , build the elementary matrix  $E$ , multiply  $U = E \cdot U$ , and fold the inverse into  $L$ . Then verify  $A = L U$ .



## Q2 (b) Cholesky decomposition

*In one sentence: for the same SPD  $A$ , find a lower-triangular  $L$  with  $A = L L^T$ .*

*The idea.*

$L$  is a ‘matrix square root’. Write out  $L L^T = A$  entry by entry and solve for each entry of  $L$ , going row by row, left to right. Two rules cover everything:

- **Diagonal:**  $L[i][i] = \sqrt{A[i][i] - \text{sum of squares of the } L \text{ entries already placed in row } i}$ .
- **Below diagonal:**  $L[i][j] = (A[i][j] - \text{sum of products } L[i][k] \cdot L[j][k] \text{ for } k < j) / L[j][j]$ .

$$\begin{array}{|c|c|c|} \hline * & 0 & 0 \\ \hline * & * & 0 \\ \hline * & * & * \\ \hline \end{array} \times \begin{array}{|c|c|c|} \hline * & * & * \\ \hline 0 & * & * \\ \hline 0 & 0 & * \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline * & * & * \\ \hline * & * & * \\ \hline * & * & * \\ \hline \end{array}$$

$A = L \text{ (lower)} \times L^T \text{ (the same numbers, mirrored into the upper triangle).}$

*Why only for SPD.*

Symmetry is what lets  $L L^T$  match  $A$ ; positive-definiteness keeps the quantity under each square root positive. Cholesky is LU specialised to SPD — about half the work, since  $L$  and  $L^T$  share entries.

## Q2 (c) QR by Gram–Schmidt

*In one sentence: turn the columns of  $A$  into perpendicular unit columns ( $Q$ ), recording the combinations in an upper-triangular  $R$ .*

*The idea.*

Process the columns left to right. Normalise the first to get  $q_1$ . For each later column, subtract its projection onto every  $q$  already built (the part ‘already covered’); what remains is perpendicular — normalise it. The projection coefficients fill  $R$  above the diagonal; the residual lengths fill  $R$ ’s diagonal.

Note: the lecture presents Gram–Schmidt through elimination on  $A^T A$ . The column-by-column projection above gives the same  $Q$  and  $R$  and is the more direct route to code — either is acceptable.

$$\begin{array}{|c|c|c|} \hline * & * & * \\ \hline * & * & * \\ \hline * & * & * \\ \hline * & * & * \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline * & * & * \\ \hline * & * & * \\ \hline * & * & * \\ \hline * & * & * \\ \hline \end{array} \times \begin{array}{|c|c|c|} \hline * & * & * \\ \hline 0 & * & * \\ \hline 0 & 0 & * \\ \hline \end{array}$$

$A \text{ (tall, } m \times n) = Q \text{ (tall, orthonormal columns)} \times R \text{ (small, upper-triangular).}$

*What to show.*

The code that builds  $Q$  and  $R$ , the result for your  $A$ , and ideally the checks  $A = QR$  and  $Q^T Q = I$  (orthonormality, Lecture 3, slide 19).

## Q2 (d) Random $7 \times 5$ , and the diagonal of $R$

*In one sentence: QR-decompose a random  $7 \times 5$  matrix and explain why the diagonal of  $R$  comes out the way it does.*

*What the shape gives you.*

$7 > 5$ , so QR applies directly. A random matrix has linearly independent columns with probability one, so Gram–Schmidt runs without breaking down.

*The observation — and the reason behind it.*

Every diagonal entry of  $R$  comes out **strictly positive**. This is not luck.  $R[j][j]$  is the *length* of the residual left after removing projections — a length, so it is  $\geq 0$ , and it cannot be 0 because a zero residual would mean that column is a combination of the earlier ones, contradicting linear independence.

- Because the diagonal is non-zero,  $R$  is invertible ( $\det R = \text{product of the diagonal} \neq 0$ ).
- A *near-zero* diagonal entry would be a warning sign — that column is nearly dependent on the earlier ones (collinearity).

*What to show.*

The random matrix,  $Q$ ,  $R$ , and a sentence or two of observation on the diagonal of  $R$ .

## A final checklist

- Entries random and computed in floating point; zero tested with a tolerance.
- No library routine does the linear-algebra work — only your own loops.
- Q1(c): size fixed at  $6 \times 9$ , entries random; use  $b = A \cdot x_{\text{seed}}$  for consistency, and verify the general solution.
- Q2(a): build the elementary matrices explicitly — do not skip to the multiplier shortcut.
- Every part: show a small hand example, the code snippet, and the output.
- One PDF, named `BITSID.pdf`; submit before the deadline; individual work.

---

The implementation and the worked examples are yours. If a step feels unclear, do it once by hand on a tiny matrix — the code is just that by-hand process written as loops.